Rhilo Sotto

COMPE470

24 November 2024

Assignment 5

Modules

Parking Lot

```
module parking_lot(
    input CLK, RST,
    input SB, SF,
    output reg ENTER, EXIT
    );

    localparam  RDY00 = 3'd0,
                IN10 = 3'd1, IN11 = 3'd2, IN01 = 3'd3,
                OUT01 = 3'd4, OUT11 = 3'd5, OUT10 = 3'd6;

    reg [2:0] present_state, next_state;

    // next state update
    always @(posedge CLK) begin
        if (RST) present_state <= 0; // synchronous LOGIC HIGH reset
        else present_state <= next_state; // update to next state
    end


    // next state combinatorial logic
    always @(*) begin
            case (present_state)
            RDY00: next_state =
                (!SB & !SF) ? RDY00:
                (!SB & SF) ? OUT01:
                (SB & !SF) ? IN10:
                (SB & SF) ?  RDY00: 3'dx;
            IN10: next_state =
```

```
            (!SB & !SF) ? RDY00:
            (!SB & SF) ? 3'dx:
            (SB & !SF) ? IN10:
            (SB & SF) ?  IN11: 3'dx;
        IN11: next_state =
            (!SB & !SF) ? 3'dx:
            (!SB & SF) ? IN01:
            (SB & !SF) ? IN10:
            (SB & SF) ?  IN11: 3'dx;
        IN01: next_state =
            (!SB & !SF) ? RDY00:
            (!SB & SF) ? IN01:
            (SB & !SF) ? 3'dx:
            (SB & SF) ?  IN11: 3'dx;
        OUT01: next_state =
            (!SB & !SF) ? RDY00:
            (!SB & SF) ? OUT01:
            (SB & !SF) ? 3'dx:
            (SB & SF) ?  OUT11: 3'dx;
        OUT11: next_state =
            (!SB & !SF) ? 3'dx:
            (!SB & SF) ? OUT01:
            (SB & !SF) ? OUT10:
            (SB & SF) ?  OUT11: 3'dx;
        OUT10: next_state =
            (!SB & !SF) ? RDY00:
            (!SB & SF) ? 3'dx:
            (SB & !SF) ? OUT10:
            (SB & SF) ?  OUT11: 3'dx;
    endcase
end

// output combinatorial logic
always @(*) begin
    case (present_state)
        RDY00:
            if (!SB & !SF) begin
                ENTER = 0; EXIT = 0;
            end
            else if (!SB & SF) begin
                ENTER = 0; EXIT = 0;
            end
            else if (SB & !SF) begin
                ENTER = 0; EXIT = 0;
```

```
    end
    else if (SB & SF) begin
        ENTER = 0; EXIT = 0;
    end
    else begin
        ENTER = 1'bx; EXIT = 1'bx;
    end
IN10:
    if (!SB & !SF) begin
        ENTER = 0; EXIT = 0;
    end
    else if (!SB & SF) begin
        ENTER = 0; EXIT = 0;
    end
    else if (SB & !SF) begin
        ENTER = 0; EXIT = 0;
    end
    else if (SB & SF) begin
        ENTER = 0; EXIT = 0;
    end
    else begin
        ENTER = 1'bx; EXIT = 1'bx;
    end
IN11:
    if (!SB & !SF) begin
        ENTER = 0; EXIT = 0;
    end
    else if (!SB & SF) begin
        ENTER = 0; EXIT = 0;
    end
    else if (SB & !SF) begin
        ENTER = 0; EXIT = 0;
    end
    else if (SB & SF) begin
        ENTER = 0; EXIT = 0;
    end
    else begin
        ENTER = 1'bx; EXIT = 1'bx;
    end
IN01:
    if (!SB & !SF) begin
        ENTER = 1; EXIT = 0;
    end
    else if (!SB & SF) begin
```

```
        ENTER = 0; EXIT = 0;
    end
    else if (SB & !SF) begin
        ENTER = 0; EXIT = 0;
    end
    else if (SB & SF) begin
        ENTER = 0; EXIT = 0;
    end
    else begin
        ENTER = 1'bx; EXIT = 1'bx;
    end
OUT01:
    if (!SB & !SF) begin
        ENTER = 0; EXIT = 0;
    end
    else if (!SB & SF) begin
        ENTER = 0; EXIT = 0;
    end
    else if (SB & !SF) begin
        ENTER = 0; EXIT = 0;
    end
    else if (SB & SF) begin
        ENTER = 0; EXIT = 0;
    end
    else begin
        ENTER = 1'bx; EXIT = 1'bx;
    end
OUT11:
    if (!SB & !SF) begin
        ENTER = 0; EXIT = 0;
    end
    else if (!SB & SF) begin
        ENTER = 0; EXIT = 0;
    end
    else if (SB & !SF) begin
        ENTER = 0; EXIT = 0;
    end
    else if (SB & SF) begin
        ENTER = 0; EXIT = 0;
    end
    else begin
        ENTER = 1'bx; EXIT = 1'bx;
    end
OUT10:
```

```
            if (!SB & !SF) begin
                ENTER = 0; EXIT = 1;
            end
            else if (!SB & SF) begin
                ENTER = 0; EXIT = 0;
            end
            else if (SB & !SF) begin
                ENTER = 0; EXIT = 0;
            end
            else if (SB & SF) begin
                ENTER = 0; EXIT = 0;
            end
            else begin
                ENTER = 1'bx; EXIT = 1'bx;
            end
        endcase
    end


endmodule
```

Counter

```
module counter(
    input CLK, RST,
    input ENTRY, EXIT,
    output reg [5:0] occupancy
    );

    always @(posedge CLK) begin
        if (RST) occupancy <= 0;
        else if (ENTRY && occupancy < 64)
            occupancy <= occupancy + 1;
        else if (EXIT)
            occupancy <= occupancy - 1;
        else if (occupancy >= 64) occupancy <= 6'dx;
    end

endmodule
```

Testbenches

Parking Lot and Counter

```
`timescale 1ns / 1ns

module tb_parking_lot;


reg CLK = 0, RST = 0, SB = 0, SF = 0;
wire ENTER, EXIT;
wire [5:0] OCP;

// 10ns period
always #2 CLK = ~CLK;

// instantiate DUTs
parking_lot PKLT(.CLK(CLK), .RST(RST), .SB(SB) , .SF(SF), .ENTER(ENTER), .EXIT(EXIT));
counter CNT(.CLK(CLK), .RST(RST), .ENTRY(ENTER), .EXIT(EXIT), .occupancy(OCP));


always @(posedge CLK) begin
RST = 1;
#20; RST = 0;

// car entering sequence
repeat(8) begin
#5; SB = 1;
#5; SF = 1;
#5; SB = 0;
#5; SF = 0;
end

#20;
repeat(8) begin
#5; SB = 1;
#5; SF = 1;
#5; SB = 0;
#5; SF = 0;
end

#20;
// car exiting sequence
```

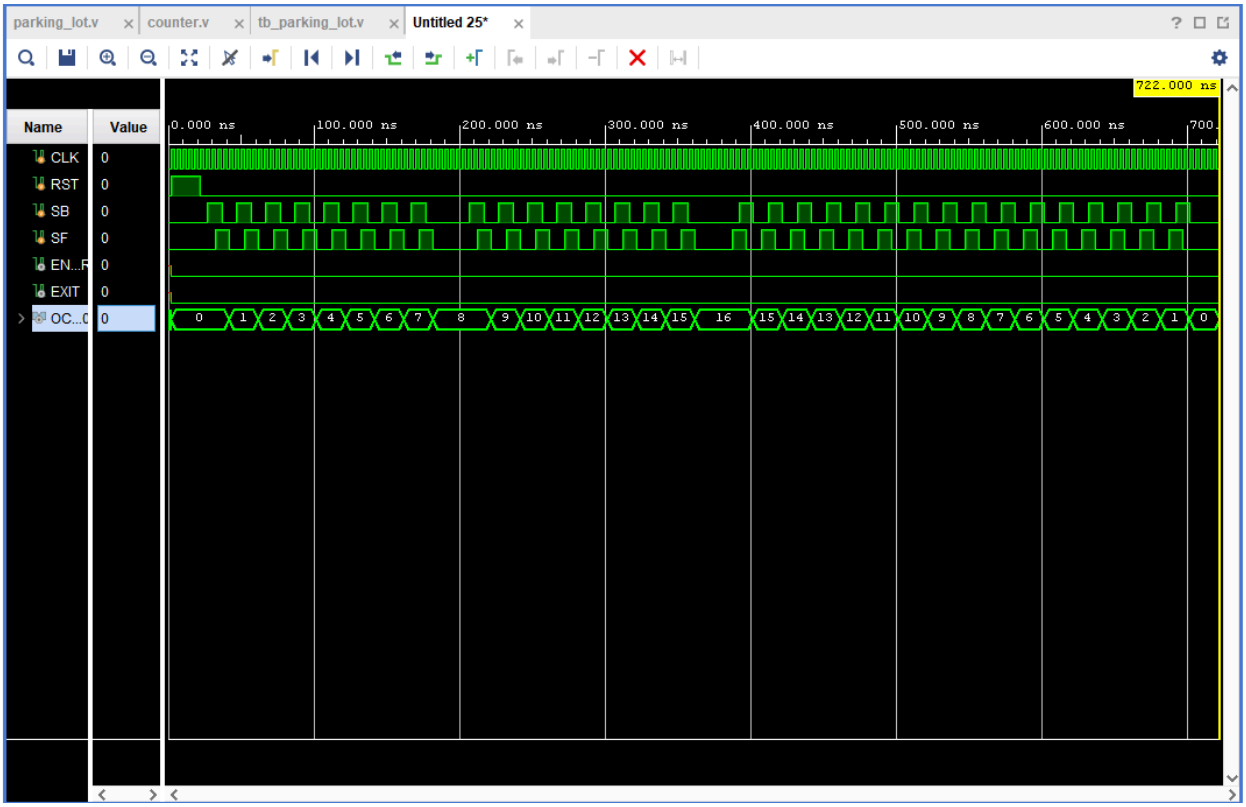
```
repeat(16) begin
#5; SF = 1;
#5; SB = 1;
#5; SF = 0;
#5; SB = 0;
end

#20;
$finish;
end

endmodule
```

Waveforms

Parking Lot and Counter



Corresponding to the parking lot testbench, the output is as expected.